

Extending a Process Mining Visualization Tool

Algorithm Enhancement (IMf), XES Support, UI Improvements, and Architectural Refactoring



AUTHOR

Marat Dussaliyev

ID: 01642049



SUPERVISOR

Dipl.-Ing. Dr.techn. Marian Lux

DATE

February 2026

Motivation & Background

PROCESS MINING CONTEXT

🔍 Bridging Data & Processes

Process mining extracts executable models from event logs to understand real operational workflows, moving beyond idealized manual modeling.

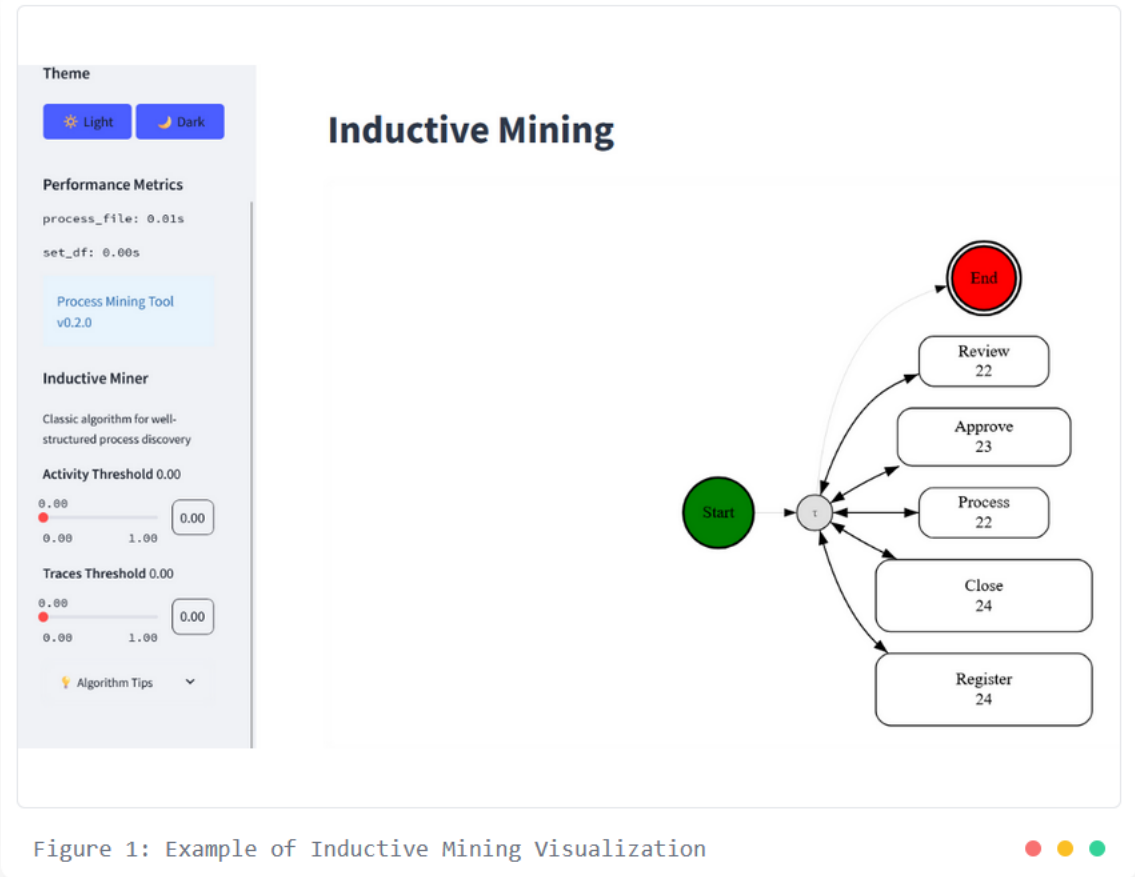
🎓 The Educational Gap

Academic settings need lightweight, interactive, web-based tools for hands-on learning without complex enterprise setups.

🔻 Handling Real-World Noise

While the standard Inductive Miner guarantees soundness, it often fails on noisy logs. The goal is to implement noise-tolerant discovery (IMf).

"Transforming raw event data into actionable process insights requires robust algorithms and accessible visualization."



Starting Point: Original Tool & Limitations

EXISTING CAPABILITIES VS. MISSING FEATURES



✓ Existing Capabilities

Mining Algorithms

Standard Inductive Miner, Heuristic Miner, Fuzzy Miner

Basic I/O & Visualization

CSV and Pickle import; interactive graph visualization; image export

⚙️ Architecture & Usability Issues

- ✗ **Tight Coupling**
UI and core algorithmic logic intermingled in a flat directory structure.
- ✗ **Poor UX**
Sparse in-app documentation; no theme system; confusing file conversion.
- ✗ **Difficult Testing**
Lack of modularity made isolated unit testing of algorithms nearly impossible.

Critical Missing Features



No XES Support

Lacked support for the IEEE standard format (eXtensible Event Stream), limiting interoperability with other process mining tools.



No Noise Tolerance (IMf)

The standard Inductive Miner fails on real-world logs with infrequent behavior, producing "flower models" instead of useful process maps.

"The tool was functional for perfect data but fragile for real-world academic use cases."

Contributions at a Glance

FIVE KEY EXTENSIONS



01

IMf Algorithm

Implemented noise-tolerant Inductive Miner Infrequent using a two-phase discovery approach (cut detection + filtering).



02

XES Support

Added native XES I/O parsing and a robust bidirectional CSV↔XES converter for interoperability.



03

UI Improvements

Designed a light/dark theme system, added in-app algorithm documentation, and interactive guidance tips.



04

Modular Architecture

Refactored monolithic code into clean core vs. UI layers with caching and testable components.



05

Validation Suite

Created comprehensive unit tests and a structural comparison tool against the PM4Py reference implementation.

IMf Concept: Handling Infrequent Behavior

NOISE TOLERANCE & FILTERING STRATEGY

Two-Phase Discovery

Attempts standard cut detection first. If only a flower model is found, infrequent DFG edges are filtered before retrying.

Edge Filtering Logic

Retains edges where $\text{freq} \geq \text{threshold} \times \text{max_freq}$. Lower thresholds keep more detail; higher simplify more.

Filtered Splitting

Traces violating operator semantics aren't discarded but assigned to the "best fitting" partition to preserve data.

Inductive Miner - Infrequent

Noise-tolerant inductive mining for process discovery

Filtering Parameters

Activity Threshold 0.00

Traces Threshold 0.00

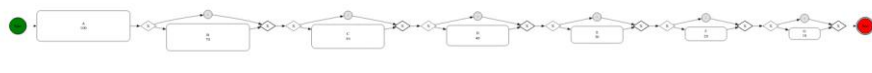
Noise Filtering

Noise Threshold 0.20

Noise Threshold Tips:

- Good for handling systematic noise while preserving structure

Back Export



Moderate Filtering

Balanced

Preserves main process structure while removing sporadic noise edges. Best for general discovery.

Inductive Miner - Infrequent

Noise-tolerant inductive mining for process discovery

Filtering Parameters

Activity Threshold 0.00

Traces Threshold 0.00

Noise Filtering

Noise Threshold 0.70

Noise Threshold Tips:

- Very aggressive - only the most frequent patterns will remain

Algorithm Tips

Back Export



Aggressive Filtering

Abstract

Only the most dominant patterns remain. Useful for simplifying complex "spaghetti" processes.

IMf Implementation Details

ALGORITHM LOGIC & SPLITTING STRATEGIES



Two-Phase Recursion

- **Phase 1:** Attempt standard Inductive Miner cuts on the full DFG.
- **Phase 2:** If no cut found (fall-through), apply filters to edges ($\text{freq} < \tau \times \text{max}$) and retry.
- **Result:** Produces a sound process tree where infrequent behavior doesn't block main structure discovery.

Filtered Splitting

Exclusive (XOR)



Assign trace to partition explaining the most events; discard others.

Sequence (SEQ)



Dynamic programming (DP) to minimize removed events across split points.

Parallel (AND)



Standard interleaving is valid; no special filtering needed.

Loop (LOOP)

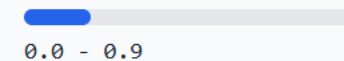
Pad invalid starts/ends with empty traces (ϵ) to maintain soundness.

```
def _inductive_mining_imf(self, log):  
    # Phase 1: Try Standard IM  
    self._use_imf_filters = False  
    if tree := self._base_cases_imf(log):  
        return tree  
  
    if res := self._try_cuts_standard(log):  
        op, sublogs = res  
        return (op, *[self._imf(sub) for sub in sublogs])  
  
    # Phase 2: Apply IMf filters if noisy  
    self._use_imf_filters = True  
    if self.noise_threshold > 0.0:  
        # Filter edges < threshold * max_freq  
        if res := self._try_cuts_filtered(log):  
            op, sublogs = res  
            return (op, *[self._imf(sub) for sub in sublogs])  
  
    return self._fallthrough_imf(log)
```

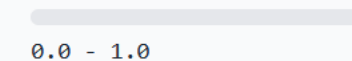
Python / Pseudocode

EXPOSED PARAMETERS

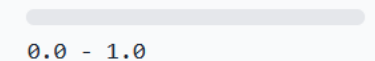
Noise Threshold



Activity Threshold



Trace Threshold



</> Native XES Parsing

Implemented using Python's standard library `xml.etree.ElementTree`. This avoids heavy external dependencies like PM4Py for basic import/export operations while ensuring standard compliance.

≡ Robust Type Handling

The parser features namespace auto-detection and correctly handles typed attributes, converting XES types (string, date, int, float, boolean) to native Python data types.

↔ Bidirectional Converter

Includes a CSV↔XES conversion workflow that preserves original attribute names (e.g., `concept:name`) for lossless round-trip operations.

- ✓ **Validation:** The BPI Challenge 2013 log was successfully converted retaining all 12 attributes across 1,487 cases and 6,660 events.

The screenshot displays the 'Convert XES to CSV Format' interface. It includes a file upload section with a 'Drag and drop file here' area and a 'Browse files' button. A file named 'BPI_Challenge_2013_closed_problems.xes' is shown. Below this, the 'CSV Output Options' section allows selecting a 'CSV Delimiter' (set to 'Comma (,)') and a checkbox for 'Include all attributes (preserve XES column names)' which is checked. A 'Convert to CSV' button is present. A blue note states: 'Note: Original XES column names preserved. This CSV can be converted back to XES without data loss.' A green success message reads: 'XES successfully converted to CSV format!'. A 'Data Summary' section lists: Columns: 12, Cases: 1487, Activities: 4, Events: 6660. A 'Data Preview' section shows a table with 12 columns: case:concept:name, concept:name, time:timestamp, org:group, resource:country, organization:country, org:resource, organization:involved, org:role, impact, product, and lifecycle:transition. The table contains 4 rows of data.

	case:concept:name	concept:name	time:timestamp	org:group	resource:country	organization:country	org:resource	organization:involved	org:role	impact	product	lifecycle:transition
0	1-109135791	Queued	2008-01-11 14:49:42+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	Awaiting Assignment
1	1-109135791	Accepted	2012-03-15 10:53:52+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	In Progress
2	1-109135791	Accepted	2012-03-15 10:56:17+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	Assigned
3	1-109135791	Accepted	2012-03-15 11:09:05+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	In Progress

Figure 4: Bidirectional File Format Converter UI

UI Enhancements: Themes & Usability

ACCESSIBILITY, DOCUMENTATION, AND USER EXPERIENCE



Theme System

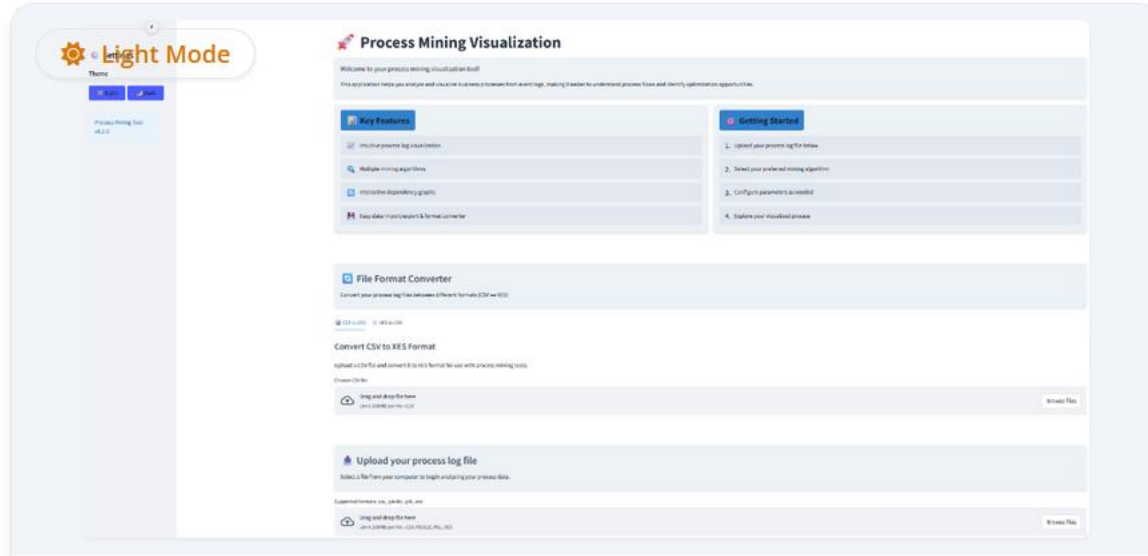
Integrated Light/Dark toggle stored in session state. Styles are injected dynamically, ensuring accessibility across different lighting conditions.

In-App Documentation

Algorithm documentation (Markdown) is rendered directly within the UI, providing immediate context on parameters like thresholds and split logic.

Workflow Guidance

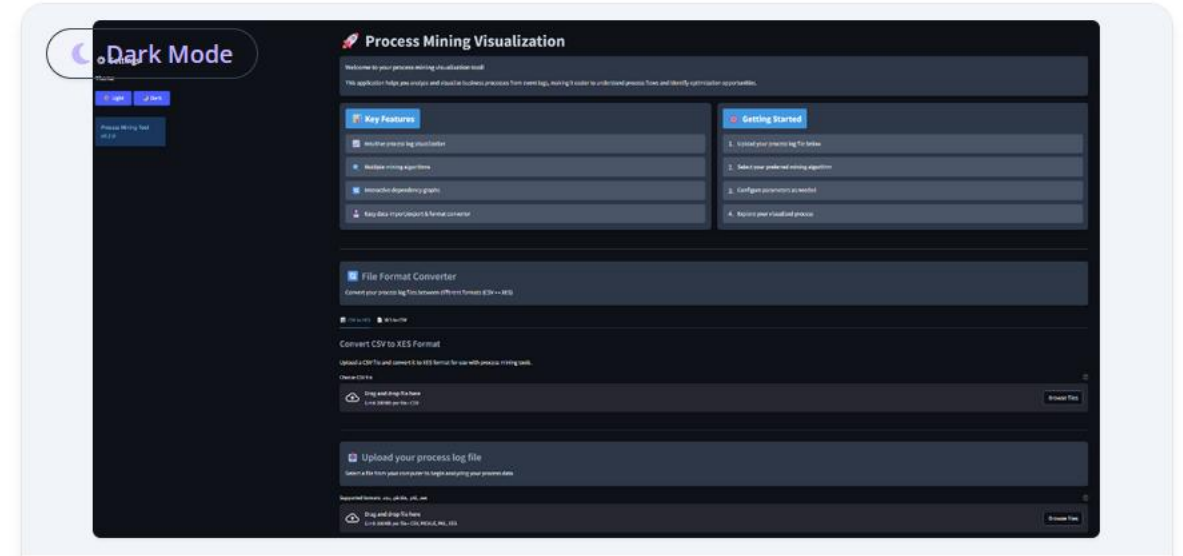
Dynamic tooltips and warnings (e.g., "Very aggressive filtering") guide users through complex parameter configurations to prevent errors.



Standard Interface

Default

High-contrast layout ideal for presentations and bright environments. Features clean typography and distinct content sections.



Dark Interface

Adaptive

Reduces eye strain during extended analysis. All components, including charts and forms, automatically adapt to the color scheme.

Architecture Refactoring

MODULARIZATION & DEPENDENCY MANAGEMENT



BEFORE Monolithic & Flat

app.py mining_algs config.py ui logger.py
graphs constants.py tests templates utils.py
assets model.py view.py controller.py data

- ✗ Flat directory structure (98 files mixed)
- ✗ UI and logic tightly coupled (circular imports)
- ✗ Difficult to test algorithms in isolation



AFTER Modular & Layered

```
src/  
├── core/ // Pure logic, No UI  
│   ├── algorithms/  
│   └── log_processing/  
├── ui/ // Streamlit components  
│   └── pages/  
├── io_ops/ // CSV, XES, Pickle  
└── tests/  
    ├── unit/  
    └── fixtures/
```

- ✓ Strict separation: Core has 0 UI dependencies
- ✓ Test hierarchy mirrors source structure

🚀 KEY TECHNICAL IMPROVEMENTS



Clear Data Flow

Unidirectional: View → Controller → Core



Performance

DFG reachability caching & Lazy imports



Testability

Mock-free unit testing of core logic

Validation & Testing

CORRECTNESS VERIFICATION & CODE COVERAGE



Comprehensive Suite

23 Unit Test Files

Covers Algorithms, Cuts, Splits, I/O, Analysis



Reference Validation

vs. **PM4Py** Implementation

Structural tree equality checked across 7 logs × 5 thresholds



Execution Status

100% Pass Rate

250 tests, 104 subtests passed in ~4.32s



```
pytest_output.log - zsh - 80x24
tests/unit/core/log_processing/sequence_log_split_test.py::TestSequenceLogSplit::test_split_with_2_partitions_
of_size_1 PASSED [ 94%]
tests/unit/core/log_processing/sequence_log_split_test.py::TestSequenceLogSplit::test_split_with_more_than_2_p
artitions PASSED [ 94%]
tests/unit/core/log_processing/test_optimal_sequence_split.py::TestOptimalSequenceLogSplit::test_example_1 PAS
SED [ 94%]
tests/unit/core/log_processing/test_optimal_sequence_split.py::TestOptimalSequenceLogSplit::test_example_2 PAS
SED [ 95%]
tests/unit/core/log_processing/test_optimal_sequence_split.py::TestOptimalSequenceLogSplit::test_example_from_
paper PASSED [ 95%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_get_xes_attributes PASSED [ 96%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_read_xes_activities PASSED [ 96%]
> ===== 250 passed, 104 subtests passed in 4.32s =====
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_read_xes_case_ids PASSED [ 96%]
```

Demo: Standard IM vs. IMf

DIRECT COMPARISON ON NOISY EVENT LOG



The Dataset

A synthetic event log containing a clear sequential process (A→B→C...) obscured by frequent random noise traces and infrequent edges.

Standard IM Result

Infrequent behavior prevents clean cuts. The algorithm falls through to the "flower model" (τ -loop), allowing any behavior (underfitting).

IMf Result

With noise threshold 0.20, infrequent edges are filtered. The algorithm successfully rediscovers the underlying sequential structure.

Standard Inductive Miner

Inductive Mining

Performance Metrics
process_file: 0.01s
set_df: 0.00s

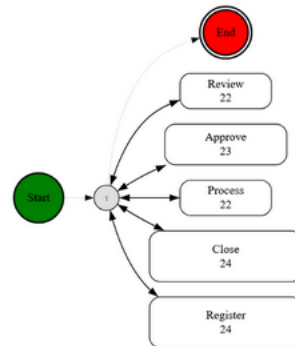
Process Mining Tool
v0.2.0

Inductive Miner
Classic algorithm for well-structured process discovery

Activity Threshold 0.00
0.00 1.00 0.00

Traces Threshold 0.00
0.00 1.00 0.00

Algorithm Tips



Flower Model (Underfitting)

The model shows a start node connected to a loop of all activities, providing no structural insight into the process flow.

No Filtering

IMf (Threshold: 0.20)

Noise-tolerant inductive mining
for process discovery

Filtering Parameters

Activity Threshold 0.00
0.00 1.00 0.00

Traces Threshold 0.00
0.00 1.00 0.00

Noise Filtering

Noise Threshold 0.20
0.00 0.20 0.99 0.00

Noise Threshold Tips:
Good for handling systematic noise while preserving structure



Discovered Sequence

A clean sequential model: Register → Review → Approve → Process → Close. The noise has been effectively removed.

Noise Filtered

Conclusion & Future Work

SUMMARY OF CONTRIBUTIONS & OUTLOOK



✓ Key Takeaways



IMf Implementation

Successfully implemented robust Inductive Miner Infrequent with filtered splitting, fully validated against PM4Py reference.



Data Interoperability

Native XES support and bidirectional CSV↔XES converter ensure lossless round-trip data handling.



Modernized Architecture

Refactored codebase with clear separation of core logic and UI, improving testability and maintainability.



Future Work



Extended Algorithms

Implementation of Inductive Miner Directly-Follows (IMd) and Alpha Miner. Adding conformance checking (token replay or alignments).



Performance Optimization

Further NumPy vectorization for large datasets, computation caching for repetitive tasks, and background processing threads.



Deployment & Extensions

Containerized deployment (Docker) for easier classroom access. Full support for XES extensions (classifiers, cost, org).

"The extended tool now serves as a robust foundation for future educational process mining research."

Refernces:

- [1] W. M. P. van der Aalst, "Process Mining: Data Science in Action," 2nd ed., Springer, 2016.
- [2] W. M. P. van der Aalst, "Process Mining: Discovery, Conformance and Enhancement of Business Processes," Springer, 2011.
- [3] IEEE Task Force on Process Mining, "XES Standard Definition," IEEE 1849-2016, 2016.
- [4] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering Block-Structured Process Models from Event Logs Containing Infrequent Behaviour," in BPM Workshops, LNBIP, vol. 171, Springer, 2014.
- [5] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering Block-Structured Process Models from Event Logs – A Constructive Approach," in Petri Nets, 2013,
- [6] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Scalable Process Discovery and Conformance Checking," Software and Systems Modeling, vol. 17.
- [7] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Process and Deviation Exploration with Inductive Visual Miner," in BPM Demos, 2014.
- [8] B. F. van Dongen, A. K. A. de Medeiros, H. M. W. Verbeek, A. J. M. M. Weijters, and W. M. P. van der Aalst, "The ProM Framework: A New Era in Process Mining Tool Support," in ICATPN, 2005
- [9] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst, "Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science," in ICPM Demo Track, 2019, pp. 13–16.
- [10] Fluxicon, "Disco – Process Mining Made Easy," <https://fluxicon.com/disco/>
- [11] M. La Rosa et al., "APROMORE: An Advanced Process Model Repository," Expert Systems with Applications, vol. 38, no. 6, pp. 7029–7040, 2011.
- [12] Streamlit Inc., "Streamlit: The Fastest Way to Build Data Apps," <https://streamlit.io/>
- [13] A. J. M. M. Weijters, W. M. P. van der Aalst, and A. K. Alves de Medeiros, "Process Mining with the HeuristicsMiner Algorithm," BETA WP 166, Eindhoven University of Technology, 2006.
- [14] C. W. Günther and W. M. P. van der Aalst, "Fuzzy Mining – Adaptive Process Simplification Based on Multi-Perspective Metrics," in BPM, 2007
- [15] W. M. P. van der Aalst et al., "Process Mining Manifesto," in BPM Workshops, LNBIP, vol. 99, Springer, 2012
- [16] S. J. J. Leemans, "Robust Process Mining with Guarantees," Ph.D. thesis, Eindhoven University of Technology, 2017.
- [17] J. Carmona, B. F. van Dongen, A. Solti, and M. Weidlich, "Conformance Checking: Relating Processes and Models," Springer, 2018.
- [18] Python Software Foundation, "xml.etree.ElementTree — The ElementTree XML API," Python Docs, <https://docs.python.org/3/library/xml.etree.elementtree.html>
- [19] Graphviz, "Graphviz – Graph Visualization Software," <https://graphviz.org/>
- [20] W. McKinney, "Data Structures for Statistical Computing in Python," in SciPy, 2010, pp. 56–61

Thank You!